

HireSense: Traffic Capacity, Crash Points & Architectural Solutions

This document serves as an engineering reference manual detailing the traffic load limits, theoretical crash points, and architectural solutions for the HireSense platform. It provides a clear blueprint for transitioning your local/MVP prototype into a highly available production application.

The Crash Threshold Matrix

System Layer	Critical Limit	Failure Condition	Consequence	Immediate Solution
Database Connections	60 simultaneous queries	Exceeding Supabase Free PostgreSQL connection cap.	500 Internal Server Error (Database connections exhausted).	Supervisor Transaction Pooling (Port 6543)
Database Storage	500 MB	Exceeding Supabase Free storage threshold.	Database locked in Read-Only Mode; new registrations/jobs fail.	Rolling Backups + Auto-Delete Policies
AI Resume Uploads	5 concurrent uploads	Overwhelming NVIDIA NIM API rate limits (e.g., Llama model racing).	429 Too Many Requests API failures; failed resume parses.	Exponential Backoff Retries + Task Queues
System Memory (RAM)	512 MB	Loading sentence-transformers locally on a free cloud runner.	OOM (Out of Memory) crash; backend process killed instantly.	Decoupled Serverless Vector Embeddings
Backend Threading	1 heavy upload	Running the NIM racing parser synchronously on the main thread.	Whole server freezes; other active users experience connection timeouts.	FastAPI Asynchronous Background Tasks

Layer-by-Layer Bottleneck Analysis

1. The Database Layer (Supabase Free)

The database is the ultimate source of truth, but on the free tier, it represents a rigid bottleneck:

- Connection Limit Math: 60 physical connections. In typical non-pooled setups, each active API worker claims a dedicated connection.
- The Crash Trigger: If you have 60 active web clients submitting requests at the exact same millisecond, the 61st query throws a connection exception.
- Storage Limit Math: 500MB is roughly 512,000 KB. A parsed candidate resume profile, including extracted text and AI metadata, takes roughly 10KB - 20KB.

$$\text{Max Candidates} = \frac{500,000 \text{ KB}}{15 \text{ KB/candidate}} \approx 33,333 \text{ candidates}$$

Once this number is reached, all new write actions are blocked.

2. The Backend Server Layer (FastAPI)

FastAPI is built on asyncio and can handle tens of thousands of idle connections. However, running CPU-heavy or network-blocking operations inside synchronous endpoint routes bypasses this advantage:

- The Crash Trigger (Blocking): If your endpoint code reads response = requests.post(...) to connect to NVIDIA NIM, that single python thread blocks completely for 3 seconds waiting for the AI. No other request can be served by that thread during that time.
- The RAM Crash Trigger: sentence-transformers is powered by PyTorch. Loading PyTorch and the BERT model (all-MiniLM-L6-v2) requires a minimum of 600MB - 800MB RAM at rest. Running this on a standard free tier cloud container (512MB limit) triggers an automatic system reboot (OOM error).

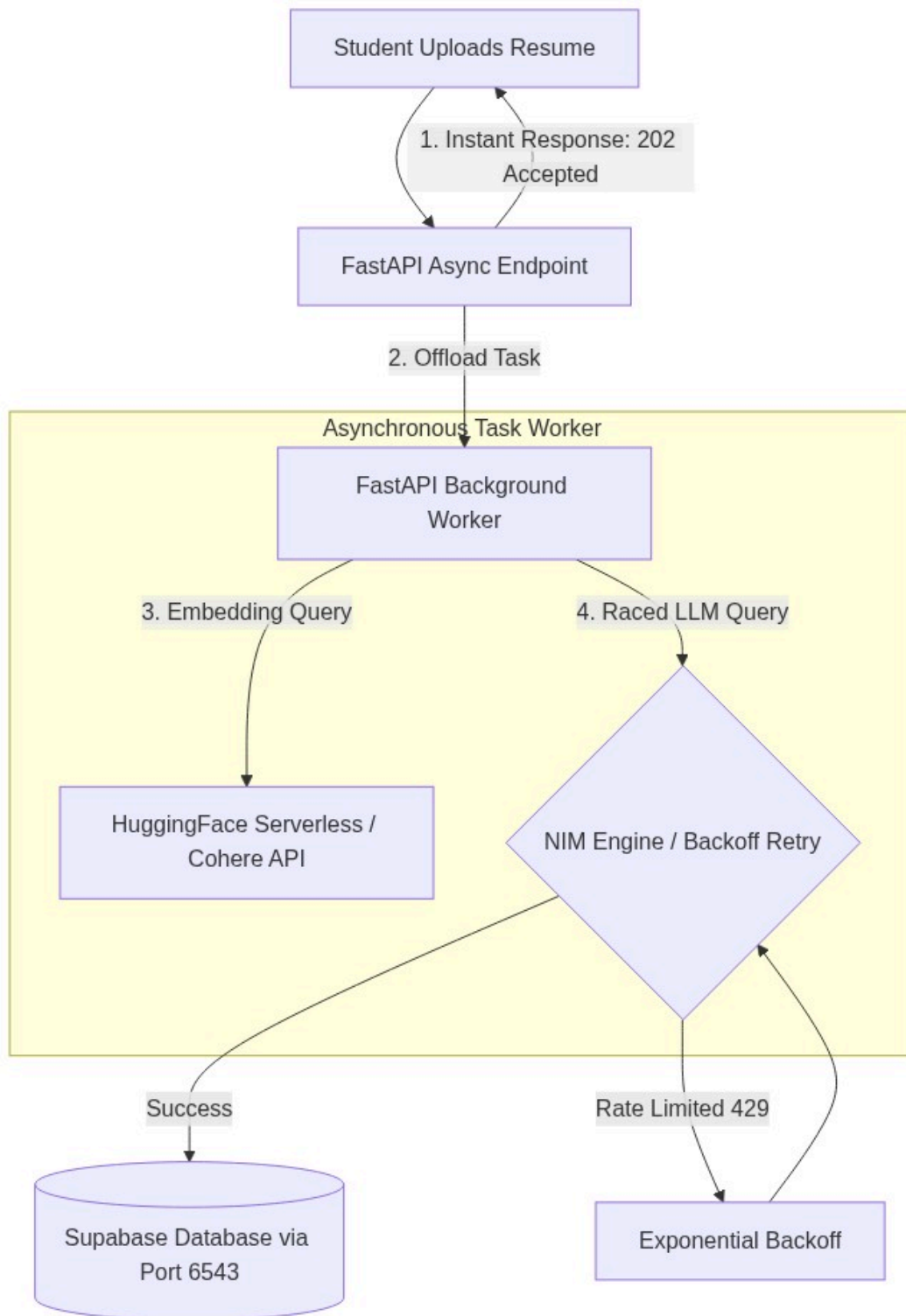
3. The AI Processing Layer (NVIDIA NIM)

Your racing engine is extremely innovative, but it multiplies traffic load:

- The Math: 1 candidate upload = 3 concurrent LLM queries (Llama-3.1-8B, Llama-3.2-3B, Llama-3.2-1B).
- The Crash Trigger: If 10 candidates apply simultaneously, your server fires 30 concurrent LLM queries within a fraction of a second. Free tier NIM credits or trial API gateways enforce sharp rate limits and will drop these calls immediately.

The Architecture Blueprint for an "Uncrashable" Platform

To scale to thousands of users, implement the following four decoupled solutions.



Solution 1: Switch to Supavisor Connection Pooling

Instead of making direct connections to the PostgreSQL engine, route traffic through the Supavisor pooling engine. This queues and reuse database connections instantly.

- Implementation: In your .env config, modify the port from 5432 to 6543 and append ?pgbouncer=true to force transaction-level pooling:

```
# BEFORE (Direct - Max 60 connections)
DATABASE_URL=postgresql://postgres.xxx:pw@aws-0.pooler.supabase.com:5432/postgres
```

```
# AFTER (Pooled - Scales to 10,000+ virtual connections)
```

```
DATABASE_URL=postgresql://postgres.xxx:pw@aws-0.pooler.supabase.com:6543/postgres?pgbouncer=true
```

Solution 2: Non-Blocking Async Background Processing

Never make the web client wait for the AI to complete its analysis. Return an immediate 202 Accepted status, and run the screening in the background.

```
from fastapi import FastAPI, UploadFile, BackgroundTasks, HTTPException
import uuid

app = FastAPI()

async def process_resume_in_background(candidate_id: str, file_content: bytes):
    """
    Background worker function that performs the heavy lifting:
    1. Extracts text from file_content
    2. Runs NVIDIA NIM Racing Engine (with rate-limit protection)
    3. Generates embeddings via API
    4. Updates database status to 'Completed'
    """
    try:
        # 1. Text Extraction
        extracted_text = extract_text_from_pdf(file_content)

        # 2. Resilient AI Race
        screening_result = await run_resilient_nim_race(extracted_text)

        # 3. Save to Supabase (Port 6543)
        update_candidate_profile(candidate_id, screening_result, status="Completed")
    except Exception as e:
        update_candidate_profile(candidate_id, None, status="Failed", error_log=str(e))

@app.post("/api/applications/apply")
async def apply_to_job(file: UploadFile, background_tasks: BackgroundTasks):
    candidate_id = str(uuid.uuid4())

    # Read file content immediately to release connection
    file_content = await file.read()

    # Create database entry with 'Processing' status
    initialize_candidate_record(candidate_id, status="Processing")

    # Queue the heavy AI workload as a background task
    background_tasks.add_task(process_resume_in_background, candidate_id, file_content)

    # Return immediately! UI updates in real-time via WebSockets later
    return {"status": "Processing", "candidate_id": candidate_id}
```

Solution 3: API-Based Embeddings (Save Server RAM)

Rather than loading PyTorch and sentence-transformers locally, offload vector generation to a free serverless

endpoint.

- The Option: Use the Hugging Face Serverless Inference API or Cohere Free Tier API to compute embeddings.
- Impact: Server RAM drops from 800MB to 50MB, allowing your FastAPI application to run flawlessly on standard free cloud hosts.

```
import httpx

async def get_serverless_embedding(text: str) -> list[float]:
    """Generates 384-dimension sentence embeddings via free Hugging Face API."""
    HF_TOKEN = "your_free_hf_api_token"
    API_URL = "https://api-inference.huggingface.co/pipeline/feature-extraction/sentence-transformers/all-MiniLM-L6-v2"

    headers = {"Authorization": f"Bearer {HF_TOKEN}"}
    async with httpx.AsyncClient() as client:
        response = await client.post(API_URL, json={"inputs": text}, headers=headers)
        if response.status_code == 200:
            return response.json()
        raise HTTPException(status_code=500, detail="Failed to fetch vector embedding")
```

Solution 4: Rate-Limit Resilient API Calls

Use exponential backoff retries when calling the NVIDIA NIM API to handle transient 429 Too Many Requests errors gracefully.

```
from tenacity import retry, wait_exponential, stop_after_attempt, retry_if_exception_type
import httpx

class RateLimitException(Exception):
    pass

@retry(
    stop=stop_after_attempt(5),
    wait=wait_exponential(multiplier=1, min=2, max=10),
    retry=retry_if_exception_type(RateLimitException)
)
async def fetch_nim_completion_with_retry(model_name: str, payload: dict):
    async with httpx.AsyncClient() as client:
        response = await client.post(
            f"https://integrate.api.nvidia.com/v1/chat/completions",
            json=payload,
            headers={"Authorization": "Bearer NIM_KEY"}
        )

        if response.status_code == 429:
            # Trigger retry mechanism with backing off delay
            raise RateLimitException("Rate limit hit, backing off...")

        if response.status_code != 200:
            raise Exception("API error occurred")

    return response.json()
```

Scalability Milestones

To maintain a smooth workflow without burning money, apply upgrades as you hit these active user markers:

[0 - 50 Users]	--> Standard Supabase Free + GitHub Backups.
[100+ Users]	--> Enable Supervisor Port 6543 + FastAPI Background Tasks.
[500+ Users]	--> Move local embeddings to HF Serverless API.
[1,000+ Users]	--> Upgrade to Supabase Pro (\$25/mo) for PITR backups.